Crypto ETL Pipeline

Automated data ingestion, validation and warehousing with Apache Airflow

Project type	Data engineering: batch ETL and workflow orchestration
Technologies	Python 3.12, Apache Airflow 2.9, PostgreSQL 16, Docker Compose, pytest
Author	R. Binny Hinn
Date	18 October 2026
Code	github.com/binnyhinn/de-pipeline

About this project
An educational and portfolio project built and tested in a local Docker environment, using a free public API and sample data. It is not a production system.

What is inside this report

1. Summary and the problem	8. Testing
2. Objectives	9. Results
3. How the pipeline works	10. Problems faced and fixes
4. Tools and technologies	11. Limitations and future work
5. Database design	12. Conclusion
6. How the code is organised	A. How to run the project
7. Docker setup	B. Glossary of terms

1. Summary and the problem it solves

This project is an automated data pipeline. Every hour it collects the latest prices of eight cryptocurrencies from a public website (the CoinGecko API), checks that the data is correct, and saves it into a database so it can be analysed later. Nobody has to run anything by hand: a tool called Apache Airflow starts the job on a schedule, and Docker packages everything so the whole system starts with one command.

Why a pipeline is needed

Data from the internet cannot be trusted as it arrives. Some values can be missing, a price can be zero or negative, the same record can appear twice, and a job that is retried after a failure can save the same data again. Doing this by hand every hour would also be slow and error-prone. This project solves these problems:

- It runs on a schedule without human effort.
- It rejects invalid records and removes duplicates.
- It keeps a copy of the original data so nothing is lost.
- It retries automatically when something fails.
- It never saves the same row twice, even when a task runs again.

> **In one sentence**
> The pipeline fetches crypto prices every hour, cleans them, and stores both the original and the cleaned data in a database.

2. Objectives

1. Collect data from a public API on an hourly schedule.
2. Use Apache Airflow to run tasks in the right order and retry failures automatically.
3. Clean and check the data in Python: reject bad values and remove duplicates.
4. Store the raw data and the cleaned data in a PostgreSQL database.
5. Package the whole system with Docker so it works the same on any computer.
6. Add logging and automated tests so the pipeline can be trusted and monitored.

Scope

Included: hourly ingestion for 8 coins (bitcoin, ethereum, solana, cardano, dogecoin, ripple, polkadot, litecoin), validation, two-layer storage, scheduling, retries, logging and unit tests.

Not included: live streaming, dashboards, failure alerts, secrets management and cloud deployment. These are listed under future improvements.

3. How the pipeline works

The pipeline is one Airflow workflow (called a DAG) named **crypto_etl**. It has four tasks. The first task fetches the data once, and then the work splits into two paths: one path saves the original response, the other path cleans the data and saves the result.

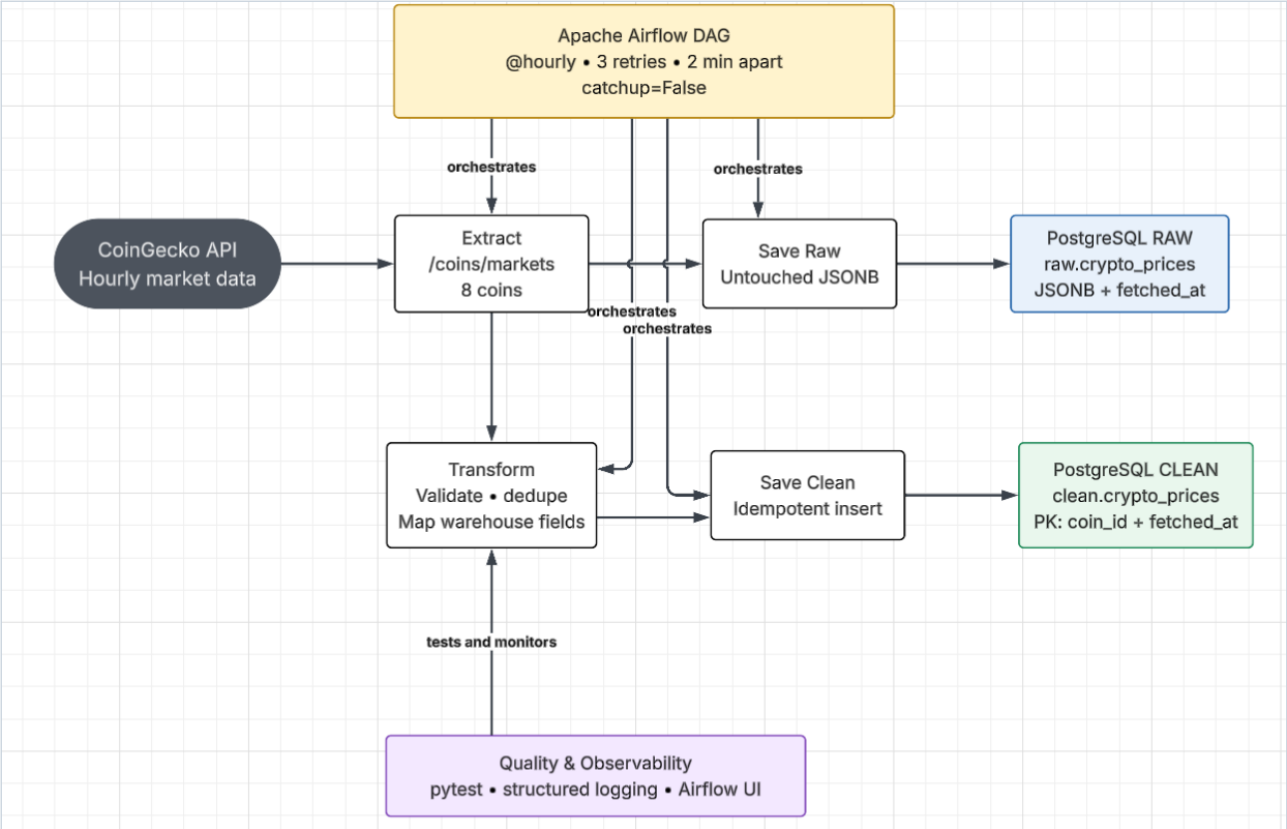


Figure 1: Data flow of the crypto_etl pipeline, from the API to the two database tables.

The four tasks, step by step

Task	What it does	Why it matters
extract	Calls the CoinGecko API and gets price, market cap, 24-hour volume and 24-hour change for 8 coins.	Gets the data. A bad response raises an error so Airflow can retry.
save_raw	Saves the untouched API response as JSON in the raw table.	Keeps the original data, so it can be reprocessed later.
transform	Rejects records with a missing id or a null, zero or negative price. Removes duplicates. Renames fields to match the database.	Makes sure only correct data goes forward.
save_clean	Inserts the validated rows into the clean table and skips any row that already exists.	Safe to repeat: retries never create duplicate rows.

Schedule and retries

Setting	Value	Meaning
schedule	@hourly	The workflow runs once every hour.
retries	3	A failed task is tried again up to three times.
retry_delay	2 minutes	Airflow waits two minutes between tries.
catchup	False	Airflow does not run all the hours it missed in the past.

4. Tools and technologies

Tool	Used for	Why it was chosen
Python 3.12	All pipeline logic	Standard language for data engineering.
Apache Airflow 2.9	Scheduling, task order, retries, monitoring	Industry standard for orchestrating data workflows.
PostgreSQL 16	The data warehouse	Reliable database with JSON support for raw data.
Docker and Docker Compose	Running everything in containers	One command starts the full system on any machine.
pytest	Automated tests	Simple and widely used test tool for Python.
CoinGecko API	Source of the data	Free and needs no API key.
VS Code and Git	Development and version control	Everyday tools for writing and tracking code.

5. Database design

The database has two layers (called schemas). Splitting data into a raw layer and a clean layer is a common data engineering practice. If the cleaning rules change later, the history can be rebuilt from the raw layer without calling the API again.

Raw layer: raw.crypto_prices

Column	Type	Description
id	SERIAL (primary key)	Row number
fetched_at	TIMESTAMPTZ	When the data was fetched
payload	JSONB	The full, unmodified API response

Clean layer: clean.crypto_prices

Column	Type	Description
coin_id	TEXT	Coin name, for example bitcoin
price_usd	NUMERIC	Current price in US dollars
market_cap_usd	NUMERIC	Market capitalisation
volume_24h_usd	NUMERIC	Trading volume in the last 24 hours
price_change_24h_pct	NUMERIC	Price change in the last 24 hours, in percent
fetched_at	TIMESTAMPTZ	When the data was fetched

> **Primary key: (coin_id, fetched_at)**
> The pair of coin and fetch time must be unique. This is what lets the pipeline skip rows that already exist, so repeated runs never create duplicates.

6. How the code is organised

The logic lives in small Python files, separate from the Airflow workflow file. This keeps every part easy to read and easy to test.

```
de-pipeline/
  dags/crypto_etl.py        the Airflow workflow (4 tasks)
  src/extract.py            calls the API
  src/transform.py          cleans and validates the data
  src/load.py               saves data into PostgreSQL
  sql/init.sql              creates the raw and clean tables
  tests/test_transform.py   unit tests
  docker-compose.yml        starts the whole system
  docs/                     report, diagrams, screenshots
```

File	What it does
extract.py	Sends the request with a 30-second timeout and checks the response status. Any error stops the task so it can be retried.
transform.py	Applies the validation rules, removes duplicates in the batch, and logs every record that is dropped.
load.py	Saves the raw JSON and the clean rows inside a transaction. Uses ON CONFLICT DO NOTHING so repeated loads have no effect.
crypto_etl.py	Defines the four tasks, their order, the hourly schedule and the retry settings.

7. Docker setup

The whole system is described in one file, docker-compose.yml, and starts with a single command: `docker compose up -d`. Docker runs five parts of the system in separate containers.

Container	Role
warehouse-db	The PostgreSQL database that holds the raw and clean tables (port 5433).
airflow-db	A separate PostgreSQL database used only by Airflow for its own records.
airflow-init	A one-time setup job. It prepares Airflow and creates the admin user, then stops.
airflow-webserver	The Airflow website where the workflow is monitored (port 8080).
airflow-scheduler	The engine that starts the workflow every hour and runs the tasks.

Folders on the computer (dags, src, tests, logs) are shared with the containers, so changes to the code appear inside Docker straight away. Health checks make sure the databases are ready before Airflow starts.

8. Testing

Automated unit tests check the cleaning rules in transform.py. They run with pytest inside the Airflow container.

Test	What it checks	Result
valid record is kept and renamed	A good record passes and its fields are renamed correctly.	Passed
missing price is dropped	A record with no price is rejected.	Passed
zero or negative price is dropped	Prices of zero or below are rejected.	Passed
missing id is dropped	A record with no coin id is rejected.	Passed
duplicates are removed	The same coin appears only once per batch.	Passed

Logging and monitoring

Every file writes log messages: how many coins were requested, how many records came back, every record that was rejected, and how many rows were saved. Airflow collects these messages for each task, and its website shows the run history, task colours, retries and failures.

9. Results

The system was started with Docker Compose and the workflow was triggered from the Airflow website. All four tasks finished successfully, shown by the green boxes in Figure 2.

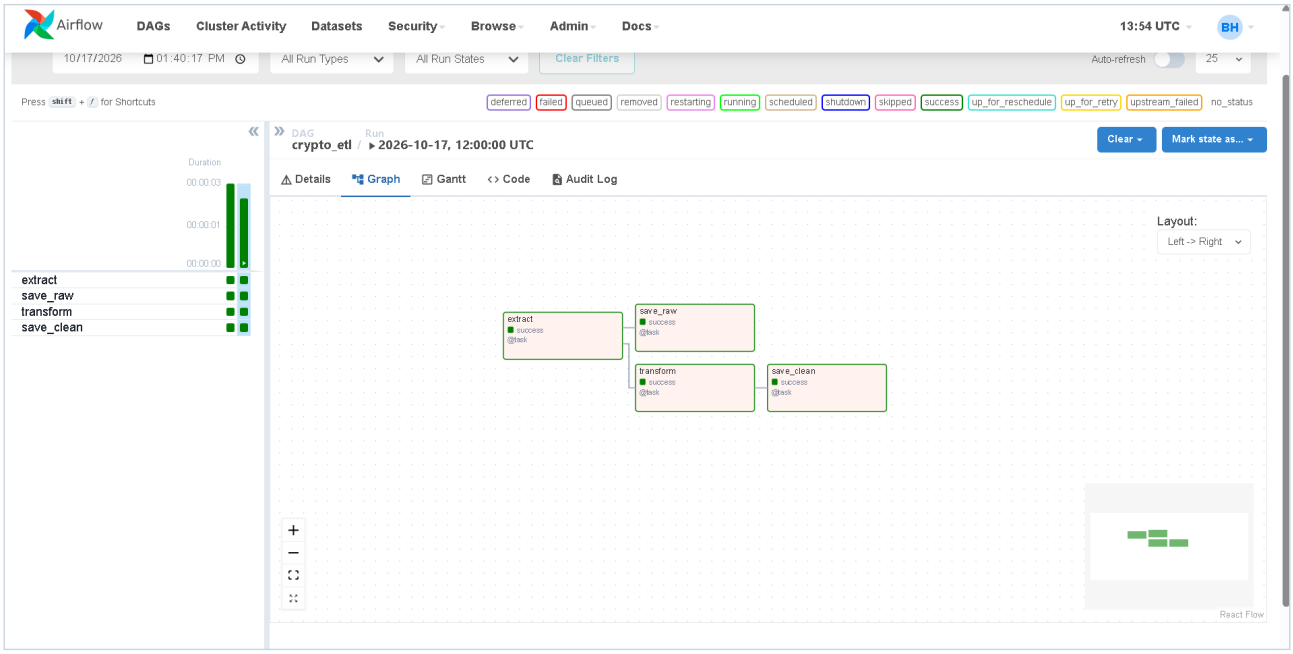


Figure 2: Airflow graph view of crypto_etl. All four tasks completed with the status success.

What the database showed

Querying the clean table returned rows for all eight coins. The table held **16 rows** instead of 8. This is correct: the workflow ran twice during testing (one manual run and one scheduled run, about twelve seconds apart), so there are 8 coins multiplied by 2 fetch times. The timestamps confirmed they were two separate runs and not duplicated data.

Check	Result
DAG run	All 4 tasks succeeded
Data loaded	Raw and clean layers both filled
Rows in clean table after 2 runs	16 (8 coins x 2 runs)
Unit tests	5 of 5 passed
Rate after setup	8 new clean rows and 1 new raw row every hour

10. Problems faced and how they were fixed

Real projects rarely work on the first try. These are the issues that came up and how each one was solved.

Problem	Cause	Solution
docker compose up failed with a Docker API error	Docker Desktop was installed but its engine was not running.	Opened Docker Desktop and waited until it showed that the engine was running.
VS Code terminal would not open	The project folder was in Restricted Mode.	Trusted the folder using the Manage option.
Airflow showed a DAG import error	load.py had an incomplete line, which caused a syntax error.	Replaced the file with the correct code, checked it on disk, and restarted the scheduler.
Table showed 16 rows, not 8	The workflow ran twice during testing.	Checked the timestamps and confirmed two separate runs.

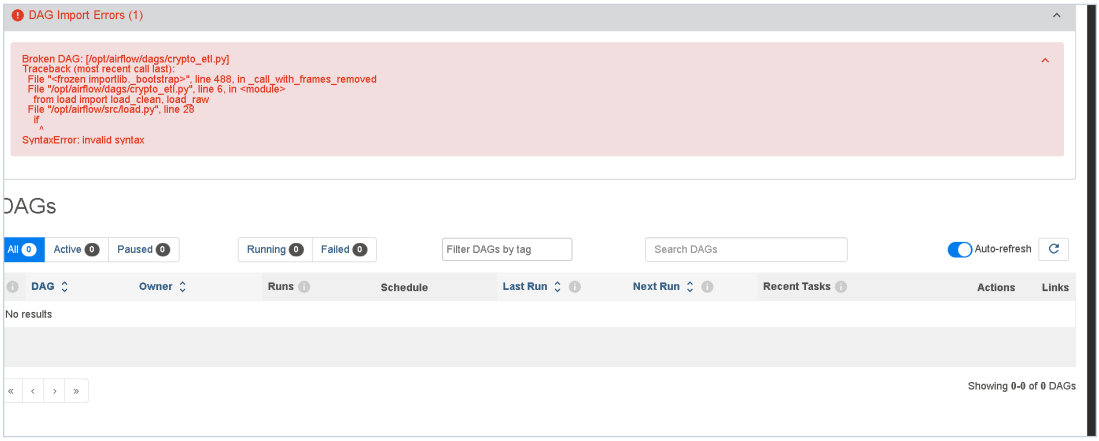


Figure 3: The Airflow import error that pointed to the exact broken line in load.py.

11. Limitations and future improvements

Limitations

- Passwords are written in the configuration files for local development. A real system would use a secrets manager.
- The free CoinGecko plan has request limits. The hourly schedule stays well within them.
- Data moves between tasks through Airflow, which suits small data but not very large datasets.

- It runs on a single machine, so it does not scale across many servers.
- Failures are visible in the Airflow website only. There are no email or chat alerts yet.
- Validation checks for missing or invalid values, not for unusual price jumps.
- Automatic retries are configured, but a forced task failure was not part of this round of testing.

Future improvements

1. Send email or Slack alerts when a task fails.
2. Add automated data quality checks, for example with Great Expectations.
3. Build a dashboard on top of the clean table to chart price history.
4. Store secrets in environment files or a secrets manager.
5. Scale out with the CeleryExecutor or Kubernetes.
6. Build the next two projects: a Kafka streaming pipeline, and a cloud warehouse with dbt and BigQuery.

12. Conclusion

This project delivers a complete, working data pipeline. It collects live data on a schedule, checks and cleans it, keeps the original next to the cleaned version, recovers from failures through retries, and avoids duplicate rows. The whole system starts with one command, the cleaning logic is covered by automated tests, and every step is logged. Together this covers the core parts of data engineering work: ingestion, orchestration, transformation, storage, reliability and monitoring.

Appendix A: How to run the project

Requirement: Docker Desktop installed and running.

Step 1: Get the project and start it

```
git clone https://github.com/binnyhinn/de-pipeline.git
cd de-pipeline
docker compose up -d
```

Step 2: Open Airflow and run the workflow

Open http://localhost:8080 and log in with admin / admin. Switch on the crypto_etl workflow and press the play button.

Step 3: Check the data

```
docker compose exec warehouse-db psql -U warehouse -d warehouse \
  -c "SELECT * FROM clean.crypto_prices LIMIT 8;"
```

Step 4: Run the tests

```
docker compose exec airflow-scheduler python -m pytest /opt/airflow/tests -v
```

Stop everything with `docker compose down`. Add `-v` to also delete the stored data.

Appendix B: Glossary of terms

Term	Simple meaning
ETL	Extract, Transform, Load: get data, clean it, save it.
Pipeline	A series of automated steps that moves data from a source to a destination.
API	A service that lets programs request data from a website.
Apache Airflow	A tool that runs tasks on a schedule, in order, and retries them if they fail.
DAG	The workflow in Airflow: a set of tasks and the order they run in.
Data warehouse	A database used to store data for analysis.
Schema	A named section of a database that groups related tables.
JSONB	A PostgreSQL column type for storing JSON data.
Primary key	A column or columns that make every row unique.
Idempotent	Safe to repeat: running it again gives the same result without duplicates.
Retry	Trying a failed task again automatically.
Docker container	A packaged mini-environment that runs one part of the system.
Docker Compose	A tool that starts several containers together from one file.